Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з лабораторної роботи №6
з дисципліни «Проектування програмних систем для мобільних пристроїв»

«Бази даних в Android»

Варіант №4

Виконав:
студент 2 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Нестерук А. О.

Дата здачі: 22.10.2026

КИЇВ – 2026

Мета. Вивчити принципи використання та керування базами даних в операційній системі Android.

Завдання. Додати у програму, розроблену у лабораторній роботі №5, функціонал, що виконує збереження всіх даних програми в БД Android та вибірку їх за потреби.

Завдання за варіантом. Доменна область: Абітурієнт.

Поля: ID, Прізвище, Ім'я, По-батькові, Адреса, Телефон, Оцінки.

Створити масив об'єктів.

Вивести:

а. список абітурієнтів, які мають незадовільні оцінки;

б. список абітурієнтів, середній бал у яких вище заданого;

в. вибрати задане число `N` абітурієнтів, що мають найбільш високій середній бал (вивести також повний список абітурієнтів, які мають напівпрохідний бал).

Хід роботи.

За основу взято застосунок з п'ятої лабораторної роботи без жодних змін до його архітектури, стеку (Rust, Tauri, React з TypeScript) чи наявних екранів. Змінено лише шар зберігання даних – файл `storage.rs`, решта модулів та усі команди залишились незмінними.

Для роботи з базою даних обрано крейт `rusqlite` – прямі Rust-прив'язки до SQLite, підключені через crates.io з функціональністю `bundled`, яка компілює SQLite з вихідного коду і тим самим дозволяє крейту збиратися під Android без додаткових налаштувань NDK. Альтернативою розглядався офіційний плагін `tauri-plugin-sql`, проте його ідіоматичне використання передбачає виклик SQL безпосередньо з фронтенду (через JS-пакет `@tauri-apps/plugin-sql`), що означало би звернення View-рівня напряму до бази даних, оминаючи рівень бізнес-логіки – класичне порушення пошарової (N-layer) архітектури, яку застосунок дотримується починаючи з другої лабораторної роботи (Controller у Rust відповідає за всю бізнес-логіку й доступ до даних, View лише викликає `invoke()`). `rusqlite` же використовується виключно на боці Rust, усередині `storage.rs`, і жодним чином не змінює цю межу.

У модулі `storage` створено таблицю `applicants` із колонками, що відповідають полям `Applicant` (`id`, `last_name`, `first_name`, `patronymic`, `address`, `phone`). Оцінки, оскільки SQLite не має типу «масив», зберігаються як текстовий рядок

через кому і розбираються назад у масив при читанні. Функція завантаження створює таблицю за потреби і заповнює її початковими даними лише тоді, коли файл бази даних не існував до відкриття з'єднання (тобто це справді перший запуск) – якщо користувач раніше видалив усіх абітурієнтів і таблиця порожня, повторне заповнення зразковими даними не відбувається. Функція збереження повністю перезаписує таблицю (видалення усіх рядків, після чого послідовні вставки поточним станом абітурієнтів) – точний аналог того, як раніше повністю перезаписувався JSON-файл.

Місце викликів команд сховища у головному модулі не змінилося: дані читаються один раз під час налаштування, а зберігаються функцією `persist` при подіях життєвого циклу Activity – `RunEvent::Exit`, `WindowEvent::Suspended` (на мобільній платформі) та `WindowEvent::CloseRequested`, із тим самим журналюванням, що й раніше. Команди додавання, редагування та видалення абітурієнта лише змінюють спільний `Repository` у пам'яті. Збереження у базу даних відбувається пізніше, при найближчій з цих подій, і в цей момент `persist()` бере повний знімок поточного стану `Repository` – тож база буде актуальною на момент зміни стану `Activity`.

Коректність перевірено на емуляторі Android: абітурієнтів додано, одного відредаговано, одного видалено, після чого застосунок згорнуто (подія `Suspended`) і перезапущено – список відображав саме той стан, у якому його залишили.

Висновок. У цій лабораторній роботі до застосунку «Абітурієнт» додано збереження всіх даних у базі даних Android замість файлового сховища у форматі JSON, яке використовувалося в попередніх роботах.

Контрольні запитання.

1. **Назвіть основні етапи встановлення бази даних MySQL.** Процес встановлення MySQL на сучасних операційних системах зазвичай розпочинається із завантаження відповідного пакета через системний менеджер пакунків, після чого відбувається розпакування та інсталяція бінарних файлів. Далі адміністратор ініціалізує системну таблицю та директорію даних за допомогою спеціальної утиліти, запускає фонову службу бази даних і обов'язково виконує скрипт безпеки для встановлення пароля користувача root, видалення анонімних користувачів та блокування віддаленого доступу з привілеями адміністратора.

2. **Назвіть основні етапи заповнення бази даних MySQL.** Заповнення бази даних розпочинається з підключення до сервера через клієнтський інтерфейс та створення нової логічної бази даних із подальшим проектуванням схеми шляхом створення таблиць із чітко визначеними типами даних і ключами. Безпосереднє додавання записів здійснюється через виконання SQL-запитів `INSERT INTO` для одиничних або пакетних вставок, або ж за допомогою утиліт масового імпорту на кшталт `LOAD DATA INFILE` для максимально швидкого завантаження великих масивів інформації з текстових файлів чи CSV безпосередньо у сховище.
3. **Опишіть довготривале зберігання даних мобільного додатка.** Довготривале зберігання даних у мобільних додатках забезпечує персистентність інформації між запусками програми або після перезавантаження пристрою шляхом запису в енергонезалежну пам'ять. Для цього розробники зберігають прості конфігураційні пари ключ-значення у спеціальних файлах, записують сирі бінарні чи текстові дані безпосередньо у внутрішнє ізольоване файлове сховище пристрою, використовують локальні реляційні бази даних для складної структурованої інформації, а також періодично синхронізують локальний стан із віддаленими хмарними серверами.
4. **Опишіть локальні бази даних та SQLite.** Локальні бази даних функціонують безпосередньо на клієнтському пристрої без розгортання окремого серверного процесу чи наявності мережевого з'єднання, забезпечуючи миттєвий доступ до структурованої інформації. Найпопулярнішим рішенням у цій ніші є SQLite – компактна реляційна система керування базами даних, написана на мові C, яка зберігає всю структуру, таблиці, індекси та дані в єдиному кросплатформному файлі на диску, повністю підтримує транзакції ACID та інтегрована на рівні платформи у більшість сучасних мобільних операційних систем.
5. **Як відбувається зберігання серій та позицій у базі даних?** Зберігання серій (наприклад, заголовків документів чи замовлень) та їхніх позицій (конкретних рядків або товарів у цих документах) реалізується через класичний зв'язок «один до багатьох» за допомогою двох окремих реляційних таблиць. Головна таблиця зберігає загальні атрибути серії, такі як унікальний ідентифікатор та статус, тоді як підлегла таблиця містить деталі кожної позиції та обов'язковий зовнішній ключ, який вказує на ідентифікатор відповідного запису в головній таблиці, що гарантує цілісність даних та дозволяє ефективно об'єднувати інформацію через SQL-оператор `JOIN`.

Лістинг.

storage.rs

```rust
//! Persisting applicants to a SQLite database in the app's data directory.
//!
//! Loaded once on startup (seeding the Lab 2 sample data if the database is
//! brand new) and saved as a full snapshot on suspend/exit, same as the
//! JSON file this replaces — only the storage backend changed.

use crate::applicant::Applicant;
use crate::sample;
use rusqlite::{Connection, params};
use std::path::Path;

fn ensure_table(conn: &Connection) -> rusqlite::Result<()> {
    conn.execute(
    "CREATE TABLE IF NOT EXISTS applicants (
    id INTEGER PRIMARY KEY,
    last_name TEXT NOT NULL,
    first_name TEXT NOT NULL,
    patronymic TEXT NOT NULL,
    address TEXT NOT NULL,
    phone TEXT NOT NULL,
    grades TEXT NOT NULL
    )",
    [],
    )?;

    Ok(())
}

fn grades_to_text(grades: &[u8]) -> String {
    grades
    .iter()
    .map(ToString::to_string)
    .collect::<Vec<_>>()
    .join(",")
}

fn grades_from_text(text: &str) -> Result<Vec<u8>, String> {
    if text.is_empty() {
        return Ok(Vec::new());
    }

    text.split(',')
    .map(|token| {
        token
        .parse::<u8>()
        .map_err(|err| format!("Invalid grade {token:?}: {err}"))
    })
    .collect()
}

fn read_all(conn: &Connection) -> Result<Vec<Applicant>, String> {
    let mut statement = conn
    .prepare(
    "SELECT id, last_name, first_name, patronymic, address, phone, grades
    FROM applicants ORDER BY id",
    )
    .map_err(|err| err.to_string())?;

    let rows = statement
    .query_map([], |row| {
```

```
        Ok((
        row.get::<_, u32>(0)?,
        row.get::<_, String>(1)?,
        row.get::<_, String>(2)?,
        row.get::<_, String>(3)?,
        row.get::<_, String>(4)?,
        row.get::<_, String>(5)?,
        row.get::<_, String>(6)?,
        ))
    })
    .map_err(|err| err.to_string())?;

    rows.map(|row| {
        let (id, last_name, first_name, patronymic, address, phone, grades) =
        row.map_err(|err| err.to_string())?;

        Ok(Applicant {
            id,
            last_name,
            first_name,
            patronymic,
            address,
            phone,
            grades: grades_from_text(&grades)?,
        })
    })
    .collect()
}

fn write_all(conn: &Connection, applicants: &[Applicant]) -> Result<(), String> {
    conn.execute("DELETE FROM applicants", [])
    .map_err(|err| err.to_string())?;

    for applicant in applicants {
        conn.execute(
        "INSERT INTO applicants
        (id, last_name, first_name, patronymic, address, phone, grades)
        VALUES (?1, ?2, ?3, ?4, ?5, ?6, ?7)",
        params![
        applicant.id,
        applicant.last_name,
        applicant.first_name,
        applicant.patronymic,
        applicant.address,
        applicant.phone,
        grades_to_text(&applicant.grades),
        ],
        )
        .map_err(|err| err.to_string())?;
    }

    Ok(())
}

pub fn load(path: &Path) -> Vec<Applicant> {
    let is_new = !path.exists();

    let result = Connection::open(path)
    .map_err(|err| err.to_string())
    .and_then(|conn| {
        ensure_table(&conn).map_err(|err| err.to_string())?;

        if is_new {
            Ok(sample::get_applicants())
```

```
        } else {
            read_all(&conn)
        }
    });

    match result {
        Ok(applicants) => applicants,
        Err(err) => {
            log::warn!(
            "Failed to load {path}: {err}. Using initial data.",
            path = path.display(),
            );
            sample::get_applicants()
        },
    }
}

pub fn save(path: &Path, applicants: &[Applicant]) -> Result<(), String> {
    let conn = Connection::open(path)
    .map_err(|err| format!("Failed to open {}: {err}", path.display()))?;

    ensure_table(&conn).map_err(|err| err.to_string())?;
    write_all(&conn, applicants)
}
```